

You want to determine the maximum memory footprint of your application. Explain how you will find out how much physical memory is being used by your application?

10. When does the Linux operating system start swapping processes to the swap device configured? How can you change this behavior?
11. Consider a case in which your user application is stuck in an infinite loop during which it invokes itself recursively. How would you prevent the stack overflow signal from killing your application?
12. Now considering the same problem in Question 11 -- instead of your user space application being stuck in an infinite loop, you have written a kernel module where a function is getting called in a recursive loop without any termination clause. Would this cause a 'Stack overflow' message?
13. Linux supports the creation of multiple file systems such as ext3, ext4 and NFS on the same physical device. Can you explain how the kernel supports multiple different file systems? For example, when a read request comes for a file located on an ext3 partition and a write request comes for a file on an ext4 partition, how does the kernel appropriately invoke the right functions?
14. You are asked to write a C code snippet which can print out the current value of the stack pointer (without invoking any assembly instructions/routines). How would you do this? Can you explain whether this is feasible? If not, explain why it may not be feasible? (Clue: think of compiler optimisations.)
15. You are running a server application which is latency-sensitive. You want to ensure that the virtual address space of the server process is locked into physical memory (essentially none of its pages should be paged out). How would you ensure this? Assume that during the course of its run, the server application can map further memory pages by dynamic memory allocations or through the *mmap* call. You need to ensure that any future pages mapped by the server also remain locked in physical memory. Explain how this can be achieved.
16. Exploring Question 15 further, given that you have locked all the pages of your server application to

remain in physical memory, is it guaranteed that your application will never get a page fault? The server application has 'mmap'd a file in to its address space and the corresponding physical address is, say, 'X'. Is it guaranteed that 'X' will never change while the application runs?

17. In the last couple of questions, we considered how a user space application can force its pages to be locked into physical memory. Now, assume you are writing a kernel module which needs to ensure that its virtual memory pages are locked in physical memory. How would you ensure this?
18. Consider that you are developing a database server application. You know from the various databases you have worked with in the past that database transactional logging can significantly add to performance overheads. Is it possible for you to eliminate the performance overhead due to logging by turning off the transactional logging in the database? If your answer is 'No,' explain why.
19. Delving deeper into Question 18, if you want to reduce the performance overhead caused by database logging (without turning off the logging completely), what are the ways to achieve this?
20. What are in-memory databases? How do they ensure that the database remains consistent in the event of a power failure or system crash?

If you have any favorite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming!  **END**

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Xerox India Research Centre. Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=HYPERLINK> <http://www.linkedin.com/groups?home=&gid=2339182> &HYPERLINK <http://www.linkedin.com/groups?home=&gid=2339182>

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

Your favourite Magazine on Open Source is now on the Web, too.

OpenSourceForU.com

Follow us on Twitter @LinuxForYou